第五周周报（4.2-4.8）

第 2 组 OS 课设周报-第 5 周-杜昊阳-富思涵-赫金科-康忆文-卢彦芸-陆昱衡-米梓润-吴安然

本周开始周报主要描述上周计划的执行情况及下周工作计划等内容

一、本周计划执行情况

1. 内存管理：杜昊阳，赫金科

- 实现五种内存分配算法 (kernel/memory.c:35-250)
  - LEGACY (0)：原有算法
  - FIRST_FIT (1)：首次适配算法
  - NEXT_FIT (2)：循环首次适配算法（使用静态变量 memman_rr_next 记录位置）
  - BEST_FIT (3)：最佳适配算法（选择最小合适块）
  - WORST_FIT (4)：最差适配算法（选择最大合适块）
- 新增算法查询功能 (kernel/memory.c:263-280)
  - memman_get_algo_id() - 返回当前算法 ID
  - memman_get_algo_name() - 返回算法名称字符串
- 新增系统调用 API (apps/apilib/api041.nas:1-15)
  - API 37 号：api_getmemalgo - 获取当前内存分配算法
- 内核头文件更新 (kernel/bootpack.h:169-190)
  - 定义算法常量：MEM_ALLOC_ALGO_LEGACY 到 MEM_ALLOC_ALGO_WORST_FIT
  - 声明新的算法查询函数
- 用户界面增强 (kernel/console.c:266-274)
  - 在 mem 命令输出中显示当前使用的算法名称

2. 中断：吴安然

- 范围明确

不修改 PIT 硬件配置（仍用通道 0，100Hz），不引入多通道；核心目标是同时存活大量软件定时器（建议 ≥50 个）时管理结构的正确性与稳定性。验收要点包括压力测试、`timer_cancel` 边界、与任务切换/键盘中断无竞态。

- 测试维度设计

设计了四个测试维度：正确性（超时顺序）、边界条件（池耗尽、取消）、并发与关中断、与任务切换交互。

- 测试计划表

制定了 TC-01~TC-09 共 9 个用例，覆盖基本超时顺序、池耗尽、大量并发定时器（≥60 个随机超时）、取消操作、与键盘中断共存、调度器时间片检查等场景。测试环境为 QEMU + 内核调试输出，记录方式为控制台日志 + 计数器。

3. 进程调度：富思涵，卢彦芸，康忆文

- 调度算法改造已完成第一阶段落地。

本周围绕进程调度主线，完成了从原有多级队列调度到 MLFQ + Aging 方案的核心迁移。实现了按队列层级动态分配时间片、运行后按行为进行降级、等待过久任务通过老化机制提升等关键逻辑，目标是兼顾系统吞吐与交互响应，降低低优先级任务长期饥饿风险

- 已形成“算法 + 运行验证”闭环。

除调度主逻辑外，本周补充了任务状态观测与验证支撑，增加了不同负载特征的任务组合用于压测与对比，包括 CPU 密集型任务、短周期交互型任务以及监控任务。通过多轮运行观察任务层级与状态变化，验证新策略在典型场景下能够按预期进行队列迁移与时间片调度。

- 阶段结果与结论。

目前调度改造已具备可运行、可观测、可复现的基础条件。初步结果表明：高负载下交互型任务响应性较改造前更稳定，低优先级任务的饥饿现象得到缓解，调度行为可通过监控任务直接验证。

4. 文件系统：米梓润，陆昱衡

- 测试程序与镜像打包

1. 新增测试应用目录：apps/memfstest（用于快速验证 memfs 全链路）
2. 关键问题与修复：
3. 链接警告 `can't link __main`：按工程 app 入口约定调整为 `_main()`。

4. 将 `mfstest.hel` 加入顶层构建与镜像打包流程（确保进入 OS 后可直接运行）。

将 memf 创建文件整体功能进行打包，使之可以直观应用

二、下周工作计划

1. 内存管理：杜昊阳，赫金科

- 在 memmap 可视化工具中显示当前内存分配算法
- 添加 mem_algo_name()函数将算法 ID 转换为可读名称
- 调用 api_getmemalgo()获取算法信息并显示在界面中

2. 中断：吴安然

- 调整 `MAX_TIMER` 至 64 或 128；编写内核测试命令（如 `test_timers`），支持同时启动多个随机超时的定时器。
- 增加调试打印（到期时输出 tick 与 data）；复用 `apps/counter` 做回归对比。
- 执行测试计划 TC-01~TC-07，记录自测结果。
- 与组内确认工作分支版本，避免合并冲突。
- 周产出：可编译运行的改动 + 自测日志 + 简短实现说明。

3. 进程调度：富思涵，卢彦芸，康忆文

- 核心目标：先把“内核态可复现竞争 + 可用同步原语”做完整，再打通用户态触发链路。
- 内核态竞争条件（两进程/多进程）

新增共享计数器/共享缓冲区场景，在无锁条件下稳定复现丢失更新。

补充两进程与多进程两组实验，记录冲突概率与错误分布

- 竞争解决方案（内核实现）

实现并验证至少一种基础同步原语（建议先自旋锁 + 关中断临界区，再扩展信号量）。

给出统一接口，支持后续同步场景复用。

- 用户态到内核态链路（系统调用）

设计最小 syscall 集：创建同步对象、P/V（或 lock/unlock）、共享区读写。

编写用户态测试程序，通过 syscall 触发竞争、切换到解决方案并验证结果。

4. 文件系统：米梓润，陆昱衡

- 研究作者使用 LDT(Local Descriptor Table)保护用户态程序的机制；基于“使用 LDT 的用户程序版本”，设计并规划展示“用户程序变量的逻辑地址到实际物理地址”的转换过程

1. 梳理工程中“用户态程序(Ring3)”的运行模型：如何装载、如何切换到用户态、如何通过系统调用回到内核态。

2. 解释 LDT 在本工程中承担的安全边界：限制用户程序只能访问其代码段/数据段，不能直接读写内核地址空间。

3. 给出“逻辑地址（段选择子:偏移）→线性地址→物理地址”的转换链路，并结合本工程的实参（LDT[0]/LDT[1] 的 base/limit、ds_base 等）说明。

4. 设计一个可演示方案：在用户程序中选取变量(全局/局部/静态/堆)，展示它们从逻辑地址到物理地址的转换与验证。